

Multi-level Queue.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
typedef struct {
```

```
    int id, at, bt, rt, ct, wt, tat,  
    type;
```

```
} process;
```

```
process p[MAX];
```

```
int n;
```

```
int sysQ[MAX], userQ[MAX];
```

```
int frontS = 0, rearS = 0;
```

```
int frontU = 0, rearU = 0;
```

```
void enqueue (int q[], int *rear,  
              int val) {
```

```
    q[( *rear )++ ] = val; }
```

```
int  
void dequeue (int q[], int *front,  
              int *rear) {
```

```
    if ( *front == *rear ) return -1;
```

```
    return q[( *front )++ ]; }
```

```
int isEmpty (int front, int rear) {  
    return q[( *front )++ ]; }
```

```
int main ( ) {
```

```
    printf ( "Enter number of processes\n" );  
    scanf ( "%d", &n );
```

```

for (int i=0; i<n; i++) {
    printf("In Process %d\n", i);
    printf("Enter arrival time: ");
    scanf("%d", &p[i].at);
    printf("Enter type (0 = system, 1 = user): ");
    scanf("%d", &p[i].typ);
    p[i].id = i;
    p[i].rt = p[i].bt;
}

```

```

for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (p[i].at > p[j].at) {
            process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

```

}
int time = 0, completed = 0, i = 0;
int current = -1;
int gantt[MAX], gtime[MAX];
gindex = 0;
while (completed < n) {
    while (i < n && p[i].at <= time) {
        if (p[i].type == 0)
            enqueue(sysQ, &rear, i);
        else
            enqueue(userQ, &rear, i);
        i++;
    }
}

```



```

if (current != -1 &&
    p[current].type == 1 &&
    !isEmpty(frontS, rearS)) {
    enqueue (userQ, rearU,
            current);
    current = -1;
}

```

```

}
if (current == -1) {
    if (!isEmpty(frontS,
        rearS))
        current = dequeue (sysQ,
            &frontS, &rearS);
    else if (!isEmpty(frontU,
        rearU))
        current = dequeue (userQ,
            &frontU, &rearU);
    else {
        time++;
        continue;
    }
}

```

```

}
gnatl[gindex] = p[current].id;
gtime[gindex] = time;
gindex++;
p[current].rt--;
time++;

```

```

if (p[current].rt == 0) {
    p[current].ct = time;
    p[current].tat = p[
    p[current].ct - p[
    & p[current].wt = p[
    .ct - p[current].bt;
    completed++;
    current--;
}
}

```

```

time [gindex] = time;
printf("In Gantt Chart\n");
for (int i = 0; i < gindex; i++) {
    printf("P%d", gantt[i]);
}

```

```

printf("\n0");
for (int i = 0; i < gindex; i++) {
    printf(" %d", gtime[i+1]);
}
float totalWT = 0, totalTAT = 0;

```

```

printf("\n\nID\tType\tWAT\t\nBT\tTAT\tWT");

```

```

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t\n");
}

```

```

p[i].id,
if (p[i].type == 0) {
    "System" : "User";
    p[i].at, p[i].bt, p[i].
}

```



```

        p[i].wt, p[i].tat);
        totalWT += p[i].wt;
        totalTAT += p[i].tat;
    }
    printf ("1.21" totalWT/n);
    printf ("1.21" totalTAT/n);
    return 0;
}

```

Output

Enter number of processes: 3

Process 0

Enter arrival time: 1

Enter burst time: 2

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 3

Enter burst time: 4

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 5

Enter burst time: 6

Enter type (0 = System, 1 = User): 0

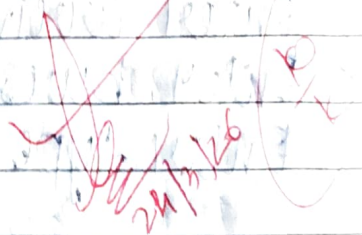
Channel Chart

P0 | P0 | P1 | P1 | P2 | P2 | P2 | P2 | P2
 0 2 3 4 5 6 7 8 9
 P2 | P1 | P1
 11 12 13

ID	Type	AT	BT	CT	WT	TAT
0	System	1	2	3	0	2
1	User	3	4	13	6	10
2	System	4	6	11	0	6

Average Waiting Time: 2.00

Average Turnaround Time: 6.00


 24/11/16

Write a program to stimulate Real-Time CPU Scheduling algorithms

- a) Rate-Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 20
```

```
typedef struct {
```

```
    int id;
```

```
    int burst;
```

```
    int deadline;
```

```
    int period;
```

```
    int share;
```

```
    int waiting;
```

```
    int turnaround;
```

```
    int completion;
```

```
} process;
```

```
void edf(process p[], int n) {
```

```
    process temp;
```

```
    float util = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        util += (float)p[i].burst /
```

```
                p[i].deadline;
```

```
    printf("In EDF Utilization = %.2f",  
           util);
```

```
printf("%d\t%d\t%d\t\t  
%d\t%d\t\n", p[i].burst, p[i].deadline,  
p[i].completion, p[i].waiting,  
p[i].turnaround);
```


TABLE 10

```

float calculate_utilization(Process p[],
    int n, int use_period){
    float utilization = 0;
    for (int i = 0; i < n; i++){
        utilization += (float) p[i].burst
            / (use_period * p[i].period
                * p[i].deadline);
    }
    return utilization;
}

```

```

void rms (Process p[], int n){
    float u = calculate_utilization(p, n, 1);
    float limit = n * (pow(2.0, 1.0/n) - 1);

    printf("In Rate Monotonic
        Scheduling (RMS)");
    printf("In Total Utilization %1.2f
        ", u, limit);

    if (u > 1.0){
        printf("IN RESULT: DEFINITELY
            NOT SCHEDULABLE IN");
        return;
    }
    else if (u > limit){
        printf("RESULT: POTENTIALLY
            NOT SCHEDULABLE IN");
    }
    else {
        printf("IN RESULT: SCHEDULABLE");
    }
}

```

33

2

```
void proportional(process p[], int n) {
    int total_shares = 0;
    for (int i = 0; i < n; i++)
        total_shares += p[i].share;
```



```

printf("In Total Shares Allocated:
%.d", total_shares);
if (total_shares > 100) {
    printf("In RESULT: OVERLOAD\n");
} else {
    printf("In RESULT: SCHEDULABLE\n");
}

```

```

int time = 0;
printf("ID BT IT share KCT KCTWT
KCTAT\n");
for (int i = 0; i < n; i++) {
    p[i].waiting = time;
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = p[i].completion;
    printf("%.d\t%.d\t%.d\t%.d\t%.d\t%.d\n",
           p[i].id, p[i].burst,
           p[i].share, p[i].completion,
           p[i].waiting, p[i].turnaround);
}
}

```

```

int main() {
    int n;
    process p[MAX], p1[MAX], p2[MAX],
           p3[MAX];
    printf("Enter number of processes:");
}

```



```

scanf("%d", &n);
for (int i = 0; i < n; i++) {
    p[i].id = i + 1;
    printf("In process %d (",
        Burst, Deadline, Period, share
        "\n", i + 1);
    scanf("%d %d %d %d",
        &p[i].burst, &p[i].deadline,
        &p[i].period, &p[i].share);
    p1[i] = p2[i] = p3[i] = p4[i];
}
rms(p2, n);
edf(p1, n);
proportional(p3, n);
return 0;
}

```

3

Output

Enter number of processes: 3

Process 1 (Burst, Deadline, period, share)
3 10 6 2

Process 2 (Burst, Deadline, period, share)
2 5 4 1

Process 3 (Burst, Deadline, period, share)
1 8 3 1

Rate Monotonic Scheduling (RMS)

Total Utilization: 1.33 (Limit: 0.78)

RESULT: DEFINITELY NOT SCHEDULABLE

Earliest Deadline First

Total Utilization: 0.83

RESULT: SCHEDULABLE

ID	BT	DL	CT	WT	TAT
2	2	5	2	0	2
3	1	8	3	2	3
1	3	10	6	3	6

P2 | P3 | P1 |

Proportional Share Scheduling

Total Shares Allocated: 4

RESULT: SCHEDULABLE

ID	BT	Share	CT	WT	TAT
1	3	2	3	0	3
2	2	1	5	3	5
3	1	1	6	5	6

P1 | P2 | P3 |

0 3 5 6

3/1/26

a) Producer - consumer using semaphore?

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
int *buffer;
int BUFFER_SIZE;
int in = 0, out = 0;
int total_items;
sem_t empty, full;
pthread_mutex_t mutex;
void * producer (void *arg) {
    for (int i = 0; i < total_items; i++) {
        int item = i + 1;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("[Producer] Produced: %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
```

```

void* consumer(void* arg) {
    for (int i = 0; i < total_items; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("[Consumer] Consumer: %.1d  

        ← Buffer [%.1d] in", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    return NULL;
}

```

```

int main() {
    pthread_t p, c;
    printf("Enter buffer size");
    scanf("%d", &BUFFER_SIZE);
    printf("Enter number of items");
    scanf("%d", &total_items);
    buffer = (int*) malloc (sizeof(int)
    * BUFFER_SIZE);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL, producer,
    NULL);

```

DATE / /
PAGE

```
pthread_create(&c, NULL,  
              consumer, NULL);  
pthread_join(p, NULL);  
pthread_join(c, NULL);  
free(buffer);  
return 0;
```

g

Output

Enter buffer size: 3

Enter number of items: 10

[Producer] Produced: 1 → Buffer[0]

[Consumer] Consumed: 1 ← Buffer[0]

[Producer] Produced: 2 → Buffer[1]

[Producer] Produced: 3 → Buffer[2]

[Consumer] Consumed: 2 ← Buffer[1]

[Producer] Produced: 4 → Buffer[0]

[Consumer] Consumed: 3 ← Buffer[2]

[Producer] Produced: 5 → Buffer[1]

[Producer] Produced: 6 → Buffer[2]

[Consumer] Consumed: 4 ← Buffer[0]

[Producer] Produced: 7 → Buffer[0]

[Consumer] Consumed: 5 ← Buffer[1]

[Producer] Produced: 8 → Buffer[1]

[Consumer] Consumed: 6 ← Buffer[2]

[Producer] Produced: 9 → Buffer[2]

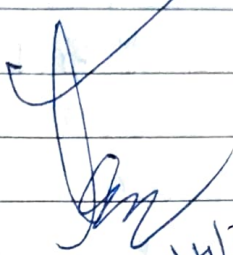
[Consumer] Consumed: 7 ← Buffer[0]

[Producer] Produced: 10 → Buffer[0]

[consumer] Consumed: 8 ← Buffer[1]

[consumer] Consumed: 9 ← Buffer[2]

[consumer] Consumed: 10 ← Buffer[0]


21/4/26

b) Dining-Philosopher Problem.

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include <semaphore.h>
```

```
#define N;
```

```
sem_t *chopstick;
```

```
sem_t room;
```

```
void* dine (void* arg) {
```

```
    int id = *(int*)arg;
```

```
    while (1) {
```

```
        printf("Philosopher %d is thinking", id);
```

```
        sleep(1);
```

```
        sem_wait(&room);
```

```
        sem_wait(&chopstick[id]);
```

```
        sem_wait(&chopstick[(id+1)%N]);
```

```
        sem_post(&room);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int i;
```

```
    printf("Enter number of philosophers");
```

```
    scanf("%d", &N);
```

```
    pthread_t philosopher[N];
```

```
    int ids[N];
```

```
    chopstick = (sem_t*) malloc (N * sizeof  
        (sem_t));
```

```
    sem_init(&room, 0, N-1);
```

```

for (i=0; i<N; i++) {
    sem_init (&chopstick[i], 0, 1);
}

for (i=0; i<N; i++) {
    ids[i] = i;
    pthread_create (&philosopher[i],
        NULL, dine, &ids[i]);
}

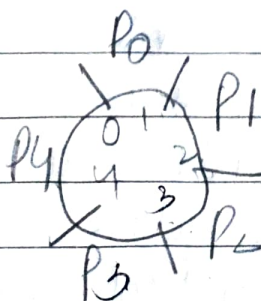
for (i=0; i<N; i++) {
    pthread_join (philosopher[i],
        NULL);
}

return 0;
}

```

Output

Step	P0	P1	P2	P3	P4
1	1	1	1	1	1
2	0	1	0	1	1
3	1	0	1	0	1
4	0	1	1	1	0
5	1	0	0	1	1
6	1	1	1	0	0
7	0	1	0	1	1
8	1	0	1	0	1



21/4/26

Output

Enter number of philosophers : 3

Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 2 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
philosopher 1 is eating
philosopher 1 finished eating
Philosopher 1 is thinking
philosopher 0 is eating
philosopher 0 finished eating
philosopher 0 is thinking
philosopher 2 is eating
philosopher 2 finished eating
philosopher 2 is thinking
philosopher 0 is eating
philosopher 0 finished eating
philosopher 1 is eating
philosopher 1 finished eating
philosopher 1 is thinking
philosopher 2 is eating
philosopher 2 finished eating
philosopher 2 is thinking
philosopher 1 is eating
philosopher 2 finished eating
philosopher 2 is eating
philosopher 2 finished eating

✓
20/12/20

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m;
    printf("Enter number of processes:");
    scanf("%d", &n);
    printf("Enter number of resources:");
    scanf("%d", &m);
    int alloc[MAX][MAX], max[MAX][MAX],
        need[MAX][MAX];
    int total[MAX], avail[MAX], work[MAX];
    int finish[MAX] = {0}, safeSeq[MAX];
    printf("\nEnter Allocation Matrix: \n");
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix: \n");
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Total Resources: \n");
    for(int j=0; j<m; j++) {
        scanf("%d", &total[j]);
    }
}
```



```

for (int j=0; j<m; j++) {
    int sum = 0;
    for (int i=0; i<n; i++) {
        sum += alloc[i][j];
    }
    avail[j] = total[j] - sum;
    work[j] = avail[j];
}

```

```

}
for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        need[i][j] = max[i][j] -
            alloc[i][j];
    }
}

```

3 3

```

printf("In Process Allocation\n");
printf("Max Need\n");
for (int i=0; i<n; i++) {
    printf("P-1.d\t", i);
    for (int j=0; j<m; j++) {
        printf("1.d\t", alloc[i][j]);
    }
    printf("\t\t");
    for (int j=0; j<m; j++) {
        printf("1.d\t", max[i][j]);
    }
    printf("\t\t");
    for (int j=0; j<m; j++) {
        printf("1.d\t", need[i][j]);
    }
    printf("\t\t");
    printf("\n");
}

```

3


```
printf("In Available");
for (int j=0; j<m; j++)
    printf("%d", avail[j]);
int count = 0;
while (count < n) {
    int found = 0;
    for (int i=0; i<n; i++) {
        if (!finish[i]) {
            int possible = 1;
            for (int j=0; j<m; j++) {
                if (need[i][j] > work[j])
                    possible = 0;
                break;
            }
            if (possible) {
                for (int j=0; j<m; j++)
                    work[j] += alloc[i][j];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (!found) {
        printf("In System is NOT SAFE (Deadlock possible)");
    }
}
return 0;
```

Output

Enter number of processes: 5

Enter number of resources: 4

Enter allocation Matrix:

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter max Matrix:

0 0 1 2

1 7 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter Total Resources:

3 14 12 12

Process	Allocation	Max	Need
P ₀	0 0 1 2	0 0 1 2	0 0 0 0
P ₁	1 0 0 0	1 7 5 0	0 7 5 0
P ₂	1 3 5 4	2 3 5 6	1 0 0 2
P ₃	0 6 3 2	0 6 5 2	0 0 2 0
P ₄	0 0 1 4	0 6 5 6	0 6 4 2

Available: 1 5 2 0

System is SAFE

Safe Sequence: P₀ → P₂ → P₃ → P₄ → P₁

28/4/20

DATE / /
PAGE

Request Resource - Allocation Algorithm.

```
int p;  
printf("Enter process no making request");  
scanf("%d", &p);  
int request[m];  
printf("Enter request vector:");  
for (int i=0; i<m; i++) {  
    scanf("%d", request[i]);  
}  
for (int i=0; i<m; i++) {  
    if (request[i] > need[p][i]) {  
        printf("ERROR: request exceeds need\n");  
        return 0; }  
}  
for (int i=0; i<m; i++) {  
    if (request[i] > avail[i]) {  
        printf("Resource not available process  
must wait\n");  
        return 0; }  
}  
for (int i=0; i<m; i++) {  
    avail[i] -= request[i];  
    alloc[p][i] += request[i];  
    need[p][i] -= request[i];  
}  
int finish[n], safeseg[n];  
for (int i=0; i<n; i++) {  
    finish[i] = 0;  
}
```


Output

Enter no of processes: 5

Enter no of resources: 3

Enter allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter max matrix:

7 5 8

3 2 2

9 0 2

2 2 2

4 8 3

Enter available resources

3 3 2

Enter process no making request: 1

Enter request vector:

1 0 2

System is in safe state

Safeseg: P1 P3 P4 P0 P2.

9/5/26

Deadlock Detection

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n, m;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources:");
```

```
    scanf("%d", &m);
```

```
    int alloc[MAX][MAX], request[MAX][MAX];
```

```
    int avail[MAX], work[MAX];
```

```
    int finish[MAX];
```

```
    printf("\nEnter allocation Matrix:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        } printf("\nEnter request Matrix:\n");
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < m; j++) {
```

```
                scanf("%d", &request[i][j]);
```

```
            } printf("\nEnter available resources:\n");
```

```
            for (int j = 0; j < m; j++) {
```

```
                scanf("%d", &avail[j]);
```

```
                work[j] = avail[j];
```

```
            }
```

```
            for (int i = 0; i < n; i++) {
```

```
                int zeroAlloc = 1;
```

```
                for (int j = 0; j < m; j++) {
```

```
                    if (alloc[i][j] != 0) {
```

```

        zeroAlloc = 0;
        break;
    }
    finish[i] = zeroAlloc;
}
int found;
do {
    found = 0;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int possible = 1;
            for (int j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }
            if (possible) {
                for (int j = 0; j < m; j++) {
                    work[j] += alloc[i][j];
                }
                finish[i] = 1;
                found = 1;
            }
        }
    }
} while (!found);
int deadlock = 0;
printf("In Deadlock Processes:");
for (int i = 0; i < n; i++) {
    if (!finish[i]) {
        printf("P%d", i);
    }
}

```



```
if (!deadlock) {  
    printf("No Deadlock");  
    printf("\n");  
    return 0;  
}
```

3

Output

Enter number of processes: 3

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

Enter Request Matrix

0 0 2

2 0 2

0 0 1

Enter Available resources:

0 0 0

Deadlock Processes: P1 P2

5/5/26

Write a C program to simulate the following
contiguous memory allocation techniques

a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void firstfit (int blocks[], int m, int processes  
int n);
```

```
void bestfit (int blocks[], int m, int processes  
int n);
```

```
void worstfit (int blocks[], int m, int processes  
int n);
```

```
void firstfit (int blocks[], int m, int processes  
int n) {
```

```
int allocation [MAX];
```

```
for (int i = 0; i < n; i++) {  
    allocation[i] = -1; }
```

```
for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (blocks[j] >= processes[i]) {
```

```
            allocation[i] = j;
```

```
            blocks[j] -= processes[i];
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
printf("In First Fit Allocation\n");
```

```
printf("Process NO. | Process Size |  
Block NO. | \n");
```

```
for (int i = 0; i < n; i++) {
```

DATE / /
PAGE

```

printf("-.d\t\t-.d\t\t", i+1,
        processes[i]);
if (allocation[i] != -1)
    printf("-.d\t\t", allocation[i]+1);
else
    printf("Not Allocated\n");
}
}

```

```

void bestfit(int blocks[], int m, int processes[],
             int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (bestIdx == -1 || blocks[j] <
                    blocks[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processes[i];
        }
    }
    printf("\nEnter Fit Allocation : \n");
}

```



```

printf("Process No. It Process size  
It Block No In");
for (int i=0; i<n; i++) {
    printf("%d\t\t%d\t\t", i+1,
           processes[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}
}

```

```

void worstfit (int blocks[], int m, int processes,
               int n) {
    int allocation[100];
    for (int i=0; i<n; i++)
        allocation[i] = -1;
    for (int i=0; i<n; i++)
        int worstIdx = -1;
        for (int j=0; j<m; j++) {
            if (blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j]
                    > blocks[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processes[i];
        }
    }
}

```

```

printf("In Worst Fit Allocation");
printf("Process No\tProcess Size\t\n");
printf("Block No.\n");
for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t", i+1,
           processes[i]);
    if (allocated[i] != 1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}
}

```

```

int main() {
    int blocks[MAX], processes[MAX];
    int m, n;
    printf("Enter number of memory blocks!");
    scanf("%d", &m);
    printf("Enter sizes of blocks:\n");
    for (int i=0; i<m; i++) {
        scanf("%d", &blocks[i]);
    }
    printf("Enter number of processes!");
    scanf("%d", &n);
    printf("Enter sizes of processes:\n");
    for (int i=0; i<n; i++) {
        scanf("%d", &processes[i]);
    }
    int blocks1[MAX], blocks2[MAX], blocks3[MAX];
    for (int i=0; i<m; i++) {
        blocks1[i] = blocks[i];
        blocks2[i] = blocks[i];
    }
}

```

```

        blocks3[i] = blocks[i];
    }
    firstfit(blocks1, m, processes, n);
    bestfit(blocks2, m, processes, n);
    worstfit(blocks3, m, processes, n);
    return 0;
}

```

Output

Enter no of memory blocks : 5

Enter size of memory blocks :

400 700 200 300 600

Enter no of processes : 4

Enter size of process : 212 512 312 526

First Fit Allocation

1	212	1
2	512	2
3	312	3
4	526	Not allocated

Best Fit Allocation

1	212	4
2	512	5
3	312	1
4	526	2

Worst Fit Allocation

1	212	2
2	512	5
3	312	1
4	526	Not allocated

Write a C program to simulate page replacement algorithms

a) FIFO

b) LRU

c) Optimal.

```
#include <stdio.h>
```

```
int findLRU (int time[], int n) {
```

```
    int i, min = time[0], pos = 0;
```

```
    for (i = 1; i < n; ++i) {
```

```
        if (time[i] < min) {
```

```
            min = time[i];
```

```
            pos = i;
```

```
        }
```

```
    }
```

```
    return pos;
```

```
}
```

```
int main() {
```

```
    int pages[30], frames[10], time[10];
```

```
    int n, i, j, k, faults, counter, pos, found;
```

```
    printf("Enter number of pages:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string:");
```

```
    for (i = 0; i < n; ++i) {
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter no of frames:");
```

```
    scanf("%d", &f);
```

// FIFO

DATE / /

PAGE

```
for (i=0; i<f; i++) frames[i] = -1;
faults = 0; pos = 0;
printf("\n
for (i=0; i<f; i++) frames[i] = -1;
faults = 0; pos = 0;
printf("\n FIFO page replacement\n");
for (i=0; i<n; i++) {
    found = 0;
    for (j=0; j<f; j++) {
        if (frames[j] == pages[i])
            found = 1;
    }
    if (!found) {
        frames[pos] = pages[i];
        pos = (pos + 1) % f;
        faults++;
    }
    printf("Page %d -> ", pages[i]);
    for (k=0; k<f; k++)
        (frames[k] != -1) ? printf("%d ", frames[k]) : printf("- ");
    printf("\n");
    printf("Total page faults : %d\n", faults);
}
```

// LRU

```
for (i=0; i<f; i++) frames[i] = -1;
faults = 0; counter = 0;
printf("\n LRU Page Replacement\n");
for (i=0; i<n; i++)
    found = 0;
```

```

for (j=0; j<f; j++) {
    if (frames[j] == pages[i];
}
found = 1;
counter++;
time[j] = counter;
break;
} if (!found) {
    if (i < f)
        frames[i] = pages[i]
    else {
        pos = find LRU (time, f);
        frames[pos] = pages[i];
    } counter++;
    time[i % f] = counter;
    fault++;
}
printf("Page %d -> ", pages[i]);
for (k=0; k<f; k++)
    (frames[k] != -1) ? printf("%d",
        frames[k]) : printf("-1");
printf("\n");
}

```

3. // Optimal

```

for (i=0; i<n; i++) frames[i] = -1;
faults = 0;
printf("In Optimal page Replacement:");
for (i=0; i<n; i++) {
    found = 0;
    for (j=0; j<f; j++)

```



```

        if (frames[j] == pages[i]) found = 1;
    if (!found) {
        if (i < f)
            frames[i] = pages[i];
        else {
            int future[10], flag[10];
            for (j = 0; j < f; j++) {
                flag[j] = 0;
                for (k = i + 1; k < n; k++) {
                    if (frames[j] == pages[k]) {
                        future[j] = k;
                        flag[j] = 1;
                        break;
                    }
                }
                if (!flag[j]) future[j] = n + 1;
            }
            int max = future[0];
            pos = 0;
            for (j = 1; j < f; j++) {
                if (future[j] > max) {
                    max = future[j];
                    pos = j;
                }
            }
            frames[pos] = pages[i];
            faults++;
            printf("Total page faults = %d", faults);
            return 0;
        }
    }
}

```

Output

Enter number of pages : 20

Enter page reference string :

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter number of frames : 4.

FIFO page Replacements:

Page 7 → 7 - - -

Page 0 → 7 0 - -

Page 1 → 7 0 1 -

Page 2 → 7 0 1 2

Page 0 → 7 0 1 2 h

Page 3 → 3 0 1 2 f

Page 0 → 3 0 1 2 h

Page 4 → 3 4 1 2 f

Page 2 → 3 4 1 2 h

Page 3 → 3 4 1 2 h

Page 0 → 3 4 0 2 f

Page 3 → 3 4 0 2 h

Page 2 → 3 4 0 2 h

Page 1 → 3 4 0 1 f

Page 2 → 2 4 0 1 h

Page 0 → 2 4 0 1 f

Page 1 → 2 4 0 1 f

Page 7 → 2 7 0 1 h

Page 0 → 2 7 0 1 h

Page 1 → 2 7 0 1 h

Total page faults (FIFO) = 10

LRU page Replacements

Page 7	→	7	-	-	-	f
Page 0	→	7	0	-	-	f
Page 1	→	7	0	1	-	f
Page 2	→	7	0	1	2	f
Page 0	→	7	0	1	2	h
Page 3	→	3	0	1	2	f
Page 0	→	3	0	1	2	h
Page 4	→	4	0	1	2	f
Page 2	→	4	0	1	2	h
Page 3	→	3	0	1	2	f
Page 0	→	3	0	1	2	h
Page 3	→	3	0	1	2	h
Page 2	→	3	0	1	2	h
Page 1	→	3	0	1	2	h
Page 2	→	3	0	1	2	h
Page 0	→	3	0	1	2	h
Page 1	→	3	0	1	2	h
Page 7	→	7	0	1	2	f
Page 0	→	7	0	1	2	h
Page 1	→	7	0	1	2	h

Total page faults (LRU) = 8

Optimal page Replacements:

Page 7	→	7	-	-	-	f
Page 0	→	7	0	-	-	f
Page 1	→	7	0	1	-	f
Page 2	→	7	0	1	2	f
Page 0	→	7	0	1	2	h

Page 3 → 3 0 1 2 f
Page 0 → 3 0 1 2 h
Page 4 → 3 0 4 2 f
Page 2 → 3 0 4 2 h
Page 3 → 3 0 4 2 h
Page 0 → 3 0 4 2 h
Page 3 → 3 0 4 2 h
Page 2 → 3 0 4 2 h
Page 1 → 1 0 4 2 f
Page 2 → 1 0 4 2 h
Page 0 → 1 0 4 2 h
Page 1 → 1 0 4 2 h
Page 7 → 1 0 7 2 f
Page 0 → 1 0 7 2 h
Page 1 → 1 0 7 2 h

Total page faults (optimal) : 8

26/5/26